

mHAVOK on Lorenz

Parallel event metrics, Chamfer distance, and channel-combo comparison

Task completed

- Standalone implementation extracted to `mhavok_lorenz.py`
- Full metrics exported for **recall, precision, accuracy, F1, mean linear R², and Chamfer distance**
- All single-, double-, and triple-channel combinations of `x`, `y`, and `z` benchmarked

Main conclusion

- For the default `x+z` setup, the best **alignment** model and best **event-F1** model are different.
- Across all channel combinations, the best event-detection model on the tested grid is `x+y+z`.

Step 1 – Building the Hankel Matrix

```
def build_hankel(signal: np.ndarray, delays: int) -> np.ndarray:
    """Stack delay-shifted copies of one channel into a Hankel matrix."""
    return np.array([
        signal[i : i + len(signal) - delays + 1]
        for i in range(delays)
    ])

# Stack all observable channels, then take the economy SVD
H = np.vstack([build_hankel(channel, delays) for channel in Y])
# H.shape = (n_channels × delays, n_timesteps - delays + 1)

_, singular_values, Vh = np.linalg.svd(H, full_matrices=False)

# Keep the top `rank` right-singular vectors as latent coordinates
V = Vh[:rank, :].T           # shape: (time_steps, rank)
```

- Each row of **H** is one observable at one time lag – the matrix encodes recent trajectory history.
- The SVD compresses that high-dimensional history into **rank** orthonormal latent modes **V**.

Step 2 – Regression for A and B

```
# Split latent coordinates into linear state and forcing
V_linear = V[:, :rank - 1] # "state" modes – expected to evolve linearly
forcing   = V[:, rank - 1]  # last mode – concentrates irregular activity

# Estimate the dynamics:  $dV/dt = A \cdot V\_linear + B \cdot forcing(t)$ 
dVdt = np.gradient(V_linear, dt, axis=0)
Theta = np.column_stack([V_linear, forcing]) # (time, rank)

model = LinearRegression(fit_intercept=False)
model.fit(Theta, dVdt)

Xi = model.coef_.T # combined operator, shape (rank, rank-1)
A = Xi[:rank - 1, :] # linear part – governs unforced evolution
B = Xi[rank - 1:, :] # forcing coupling – maps f(t) → state tendency
```

- The regression is ordinary least squares with **no intercept** – the mean is already removed by the SVD.
- **A** is the reduced linear operator; eigenvalues of **A** should have slightly negative real parts.
- **B** quantifies how much each state mode is driven by the forcing coordinate.

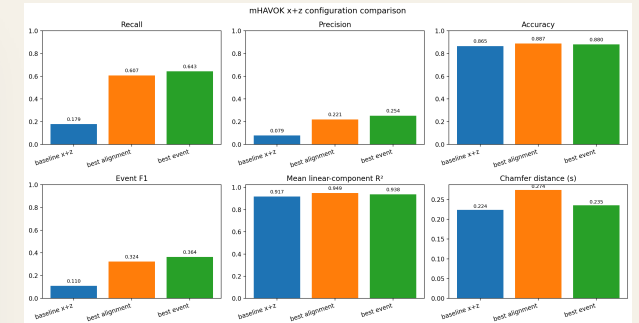
x+z Baseline vs Tuned mHAVOK Models

Baseline and winners

Model	Delays	Rank	Recall	Precision	Accuracy	F1	Mean linear R ²	Chamfer (s)
Baseline x+z	100	9	0.1786	0.0794	0.8653	0.1099	0.9173	0.2236
Best alignment	150	13	0.6071	0.2208	0.8872	0.3238	0.9493	0.2740
Best event model	150	11	0.6429	0.2535	0.8801	0.3636	0.9382	0.2352

Takeaway

- The tuned x+z model clearly improves event metrics over the baseline.
- The **best alignment** model is not the same as the **best event-**



Direct comparison of baseline, best-alignment, and best-event x+z models on the requested metrics.

Step 3 – Thresholding and Event Detection

```
forcing_threshold = np.quantile(np.abs(forcing), 0.95)
active_mask = np.abs(forcing) >= forcing_threshold
onset_indices = np.where(np.diff(active_mask.astype(int)) == 1)[0] + 1
event_times = time_havok[onset_indices]
matched_switches, _ = match_event_times(switch_times, event_times, tol=0.10)
tp = matched_switches.sum()
recall = tp / len(switch_times)
precision = tp / len(event_times)
f1 = 2 * recall * precision / (recall + precision)
true_labels = compute_temporal_labels(time_havok, switch_times, tol=0.10)
accuracy = np.mean(true_labels == active_mask)
```

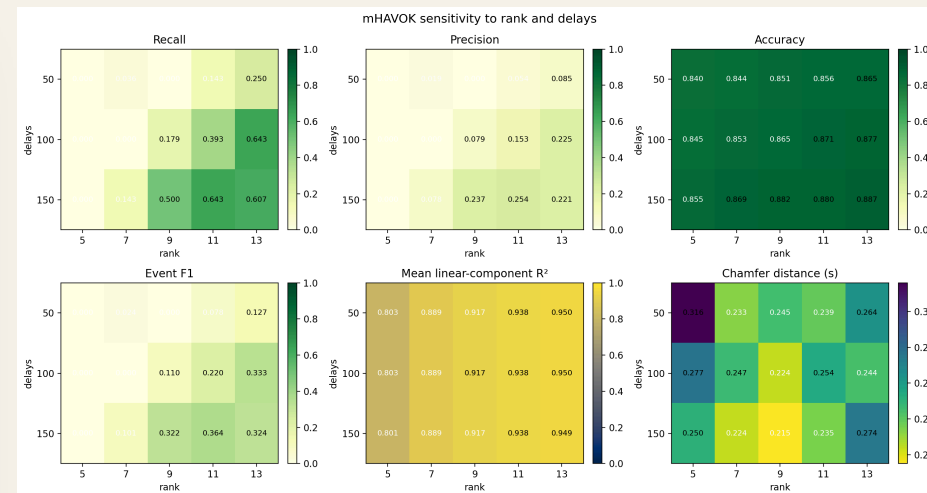
Rank-Delay Sweep for Event Metrics

Best x+z settings on the tested grid

Objective	Delays	Rank	Recall	Precision	Accuracy	F1	Mean linear R^2	Chamfer (s)
Best event F1	150	11	0.6429	0.2535	0.8801	0.3636	0.9382	0.2352
Best alignment	150	13	0.6071	0.2208	0.8872	0.3238	0.9493	0.2740

Reading the heatmap

- Event metrics improve strongly at **higher delays**.
- High mean linear R^2 does not uniquely determine the best event model.
- Chamfer distance and F1 do not select the same point as tight alignment.



Recall, precision, accuracy, F1, mean linear R^2 , and Chamfer distance over the tested rank-delay grid.

Step 4 – Chamfer Distance

```
def chamfer_distance_1d(
    ref: np.ndarray,
    cand: np.ndarray,
) -> float:
    """Symmetric nearest-neighbour distance between two 1-D event sets."""
    # Build the full pairwise  $|\Delta t|$  matrix ( $n_{\text{ref}} \times n_{\text{cand}}$ )
    D = np.abs(ref[:, None] - cand[None, :])

    # Each reference event finds its nearest prediction, and vice versa
    return 0.5 * (
        np.mean(D.min(axis=1)) # ref → nearest pred
        + np.mean(D.min(axis=0)) # pred → nearest ref
    )
```

```
chamfer = chamfer_distance_1d(switch_times, event_times)
# Lower is better – penalises large timing gaps that F1 can miss
```

- **F1** counts whether events are matched within a fixed tolerance window – a near-miss and a 10 s miss both count the same way.

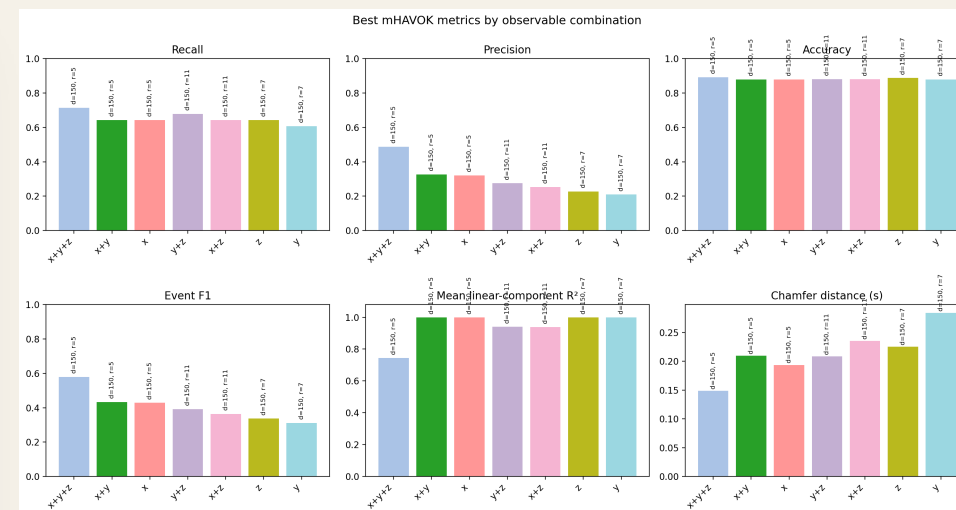
Channel Combinations: x, y, z Matter

Best model from each channel combination

Combo	Delays	Rank	Recall	Precision	Accuracy	F1	Mean linear R ²	Chamfer (s)
x+y+z	150	5	0.7143	0.4878	0.8924	0.5797	0.7450	0.1488
x+y	150	5	0.6429	0.3273	0.8794	0.4337	0.9988	0.2095
x	150	5	0.6429	0.3214	0.8792	0.4286	0.9995	0.1937
x+z	150	11	0.6429	0.2535	0.8801	0.3636	0.9382	0.2352

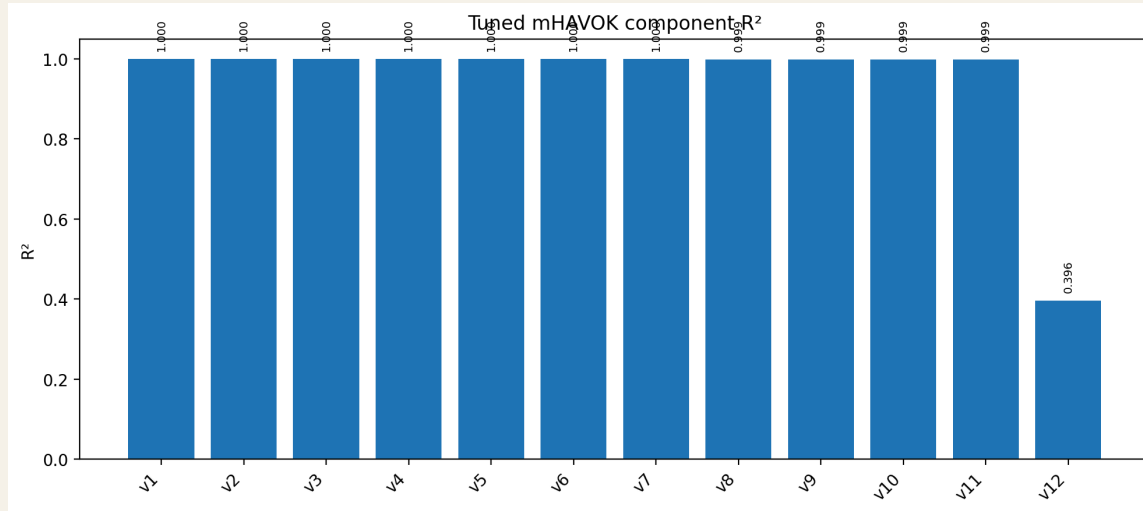
Main result

- The strongest event-detection model on the tested grid is **x+y+z**.
- Adding **y** materially changes the ranking.



Per-combo best recall, precision, accuracy, F1, mean linear R², and Chamfer distance.

R² Diagnostics and Objective Tradeoffs



Tuned x+z component R² values: most modes are near-perfect, but the last mode is much weaker.

Tuned x+z component fit

- v1 through v7: **1.0000**
- v8: **0.9991**
- v9: **0.9989**
- v10: **0.9992**
- v11: **0.9986**
- v12: **0.3957**

Observable-selection winners from the smaller objective table

Objective	Winner
Tight alignment	z only
Balanced	x and z
Early warning	x only

Interpretation

- Objective choice matters even before the full channel-combo benchmark.

Final mHAVOK Takeaways

What is now in the code

- Standalone manual mHAVOK implementation in `mhavok_lorenz.py`
- Exported CSV summaries and slide-ready figures in `plots/mhavok_lorenz`
- Dedicated metrics for **recall, precision, accuracy, R^2 , and Chamfer distance**

Scientific conclusion

- The best mHAVOK model depends on the objective.
- For the full channel benchmark, **x+y+z** is the best event-detection model on the tested grid.
- For the default x+z setup, **best alignment** and **best event F1** are different configurations.

BEST X+Z DELAYS, RANK

150, 11

BEST COMBO DELAYS, RANK

150, 5

BEST COMBO CHAMFER

0.149 s